

```

#include <iostream>
#include <string>
#include <vector>
#include <cstdint>
#include <iomanip>

#define OPCODE_RV_SVE 0b1011011
#define FUNCT3_VMUL_VV 0b010
#define FUNCT7_VECTOR 0b0000000

#define RV_SVE_ENCODE(f7, rs2, rs1, f3, rd, opc) \
(((uint64_t)(f7) << 25) | ((uint64_t)(rs2) << 20) | ((uint64_t)(rs1) << 15) | \
((uint64_t)(f3) << 12) | ((uint64_t)(rd) << 7) | ((uint64_t)(opc)))

using namespace std;

// ===== RV-SVE 명령어 구조 =====
struct Uop {
    uint64_t raw;
    int rd, rs1, rs2, funct3;
    bool is_vector;
    string opcode;

    void decode(uint64_t inst) {
        raw = inst;
        rd = (inst >> 7) & 0x1F;
        rs1 = (inst >> 15) & 0x1F;
        rs2 = (inst >> 20) & 0x1F;
        funct3 = (inst >> 12) & 0x7;
        is_vector = ((inst & 0x7F) == OPCODE_RV_SVE);
        opcode = (is_vector && funct3 == FUNCT3_VMUL_VV) ? "VMUL.VV" : "NOP";
    }
};

// ===== 데이터 버스 =====
class DataBus {
public:
    void transmit(const string& src, const string& dst, const Uop& uop, const string& tag) {
        cout << "[Bus] " << src << " → " << dst << " | 0x" << hex << setw(16) << setfill('0') <<
uop.raw
        << " [" << tag << "]" << endl;
    }
};

// ===== X코어 =====
class XCore {
public:
    string name;

```

```

XCore(const string& n) : name(n) {}

Uop generate_vector_uop() {
    uint64_t inst = RV_SVE_ENCODE(FUNCT7_VECTOR, 2, 1, FUNCT3_VMUL_VV, 0,
    OPCODE_RV_SVE);
    Uop uop;
    uop.decode(inst);
    cout << "[" << name << "] Generated: " << uop.opcode << endl;
    return uop;
}
};

// ===== A코어 =====
class ACore {
public:
    string name;
    ACore(const string& n) : name(n) {}

    Uop generate_scalar_uop() {
        uint64_t inst = 0x002081B3; // ADD r3 = r1 + r2
        Uop uop;
        uop.decode(inst);
        cout << "[" << name << "] Generated: " << uop.opcode << endl;
        return uop;
    }
};

// ===== GPU =====
class GPU {
public:
    string priority, token;
    Uop uop;

    void process(const Uop& in) {
        uop = in;
        cout << "[GPU] Decode: " << uop.opcode << " r" << uop.rd << " <- r" << uop.rs1 << " *
r" << uop.rs2 << endl;
        int load = (uop.rs1 + uop.rs2) * 64;
        priority = (load > 4096) ? "high" : "low";
        token = "DISPATCH_" + uop.opcode + "_" + (priority == "high" ? "FAST" : "IDLE");
        cout << "[GPU] Token: " << token << endl;
    }

    Uop get_uop() const { return uop; }
};

// ===== NPU =====
class NPU {

```

```

public:
    static constexpr const char* SEGMENTS[9] = {
        "Idle", "Conv2D", "MatMul", "ReLU", "Softmax", "BatchNorm", "DepthwiseConv", "GRU",
        "Attention"
    };

    void process(const Uop& uop, const string& hexCode, int h, int w) {
        int segID = stoi(hexCode, nullptr, 16);
        string op = SEGMENTS[segID];
        string priority = (h * w > 4096) ? "high" : "mid";
        string token = "EXEC_" + op + "_" + (priority == "high" ? "FAST" : "IDLE");

        cout << "[NPU] " << token << " dispatched for " << op << " (" << h << "x" << w << "
tensor)" << endl;
        cout << "[RV-SVE] " << uop.opcode << " r" << uop.rd << " <- r" << uop.rs1 << " * r" <<
uop.rs2 << endl;
    }
};

// ===== 전체 시스템 시뮬레이션 =====
int main() {
    DataBus bus;
    GPU gpu;
    NPU npu;

    // CPU 코어 생성
    vector<XCore> xcores = { XCore("XCore0"), XCore("XCore1") };
    vector<ACore> acores = {
        ACore("ACore0"), ACore("ACore1"), ACore("ACore2"),
        ACore("ACore3"), ACore("ACore4"), ACore("ACore5")
    };

    // X코어 → GPU
    for (auto& x : xcores) {
        Uop xuop = x.generate_vector_uop();
        bus.transmit(x.name, "GPU", xuop, xuop.opcode);
        gpu.process(xuop);
        bus.transmit("GPU", "NPU", gpu.get_uop(), "TensorOp");
        npu.process(gpu.get_uop(), "0x02", 128, 128); // MatMul
    }

    // A코어 → GPU (스칼라 명령어는 GPU에서 무시됨)
    for (auto& a : acores) {
        Uop auop = a.generate_scalar_uop();
        bus.transmit(a.name, "GPU", auop, auop.opcode);
        // GPU는 벡터 명령어만 처리하므로 생략 가능
    }
}

```

```
    return 0;  
}
```